PHASMDAG: Bi-Objective, Closure-Aware DAG Consensus

Towards a Separation of Order, Closure, and Finality

in High-Throughput Proof-of-Work Consensus

Arthur Zhang

April 2026

Abstract

This paper sets out a preliminary formulation of *PHASMDAG*, a bi-objective, closure-aware DAG consensus protocol intended to separate three semantic roles that remain conflated in existing GhostDAG-family designs: *ordering*, *closure maturity*, and *finality*. Recent work by Wu, Wei, Zhang, and Jiang (NDSS 2024) suggests that the *Decoupling Assumption*—a prerequisite for the security analyses of the major DAG-based PoW protocols—may fail under sufficiently high throughput, owing to Block Jam and Late-Predecessor effects. The present paper proceeds from the contention that the underlying difficulty in GhostDAG-family protocols may lie not only in the vulnerability of relay paths, but also in the demand placed upon a single structural classifier to stand simultaneously for canonical order, safe acceptability, and finality confidence. PHASMDAG accordingly treats closure as a first-class protocol state and studies a bi-objective consensus functional $\Phi(S) = U_{\text{order}}(S) - \lambda\, R_{\text{closure}}(S)$ that trades structural concurrency against dependency fragility. In the proposed v_0 instantiation, the closure layer is expressed through three deterministic sub-metrics—dependency completeness, descendant-support coherence, and frontier stability—whose aggregation and penalty terms are rendered protocol-visible. The paper develops this protocol sketch, records a preliminary security heuristic under the Congestible Blockchain Model, and isolates *closure coherence* as the principal unresolved theorem target for any consistent notion of strong confirmation.

Keywords: DAG consensus; GhostDAG; closure-aware consensus; late predecessor; block jam; bi-objective optimisation; finality semantics

1 Introduction

DAG-based Proof-of-Work consensus protocols—SPECTRE [4], PHANTOM/GhostDAG [5, 6], OHIE [9], and Prism [10]—represent the most promising direction for scaling Nakamoto consensus beyond the throughput ceiling imposed by chain-style protocols. By allowing blocks to reference multiple parents and thereby capture concurrency, these protocols achieve block rates that far exceed the network delay bound D.

However, Wu et al. [7] proved that this performance gain comes with a previously unaccounted security cost. Their *Congestible Blockchain Model* (CBM) shows that when block production rate f is high relative to network capacity, two phenomena emerge:

1. **Block Jam** (BJ): when the total size of priority blocks in a time window exceeds the network's relay capacity, these blocks cannot propagate within the assumed delay D.

2. **Late Predecessor** (LP): a block's propagation completion does not imply its acceptability—a block can only be accepted after *all* its predecessors have been received and validated. If a predecessor arrives later than its successor, the successor's actual delay ($\Delta_{\text{actual}} = \Delta_{\text{prop}} + \Delta_{\text{proc}}$) exceeds its propagation delay alone.

Together, BJ and LP violate the *Decoupling Assumption* [7]: the assumption that priority blocks can propagate within fixed short delay D *and* be immediately accepted and built upon. All existing security proofs for DAG-based PoW protocols rely on this assumption.

Contribution summary.

1. **Root-cause diagnosis.** It is argued that the fundamental limitation of GhostDAG-family protocols lies not merely in the attackability of relay paths, but in the requirement that a single structural classifier should simultaneously represent canonical order, safe acceptability, and finality confidence. This conflation is largely invisible under optimistic network assumptions, but becomes a security liability under CBM.
2. **Protocol design.** A candidate protocol design is set out in which order, closure maturity, and finality are separated into three explicit semantic layers. The proposal introduces closure as a first-class protocol state together with a bi-objective consensus functional $\Phi(S) = U_{\text{order}}(S) - \lambda\, R_{\text{closure}}(S)$ that penalises dependency fragility. For v_0, a GhostDAG-style blue-work lineage is taken as the canonical order core, while closure is instantiated through the deterministic sub-metrics $\mathcal{C}_{\text{dep}}(B)$, $\mathcal{C}_{\text{coh}}(B)$, and $\mathcal{C}_{\text{fr}}(B)$.
3. **Preliminary security analysis.** A preliminary closure-modulated chain-growth heuristic is recorded under the CBM, together with a candidate dominance region in which closure awareness may outperform purely structural finality under LP attack. Claims advanced here are explicitly separated from claims that remain to be established.
4. **Formal modelling and verification roadmap.** A compositional formal model, a simulation-based evaluation methodology, and a phased verification programme are delineated around the theorem target of closure coherence.

On the designation "PHASM". The designation *PHASMDAG* is intended to evoke *phase-separated semantics*: order, closure, and finality are treated as distinct protocol phases rather than being collapsed into a single structural score.

Claim status. This document is intentionally written as a protocol proposal rather than as a completed proof artifact. **Definitions** specify protocol intent. **Lemmas** and **Propositions** are preliminary and hold only under the modelling assumptions stated around them. **Conjectures** identify unresolved validation targets. The most important unresolved target is *closure coherence*: whether $\mathcal{C}(B)$ converges sufficiently across honest nodes to support consistent strong confirmation.

2 Related Work and Theoretical Context

2.1 Throughput, Security, and Delay in PoW Consensus

The tension between throughput and network delay predates DAG-based consensus. Sompolinsky and Zohar's high-rate Bitcoin analysis [1] already emphasised that short inter-block intervals and bounded delay cannot be treated independently. Formal longest-chain analyses subsequently made this dependence explicit through common-prefix, chain-growth, and chain-quality style guarantees

under bounded-delay assumptions [2]. Zhang and Preneel [8] later unified these perspectives into a common-metrics framework, making it possible to compare protocol families under aligned security notions rather than under protocol-specific metrics.

Two considerations may be taken from that literature. First, performance claims in PoW systems are meaningful only relative to an explicit delay model. Second, security arguments become fragile when propagation, acceptance, and mining eligibility are silently conflated.

2.2 DAG-Based PoW Ordering Protocols

The DAG-based PoW literature can be organised by the manner in which concurrency is converted into order:

SPECTRE [4] provides fast confirmation through pairwise voting, but does not aim to maintain a global total order.

PHANTOM [5] formulates ordering as a k-cluster identification problem, thereby making explicit the relation between concurrency and anticone structure.

GhostDAG [6] replaces exact clustering with a greedy K-cluster approximation, yielding a practical structural order rule based on blue-set growth and blue-work accumulation.

OHIE [9] separates transaction handling from structural ordering through a numbered-chain construction.

Prism [10] decomposes consensus into proposal, voting, and committee roles, thereby achieving high throughput through role separation rather than through a single ordering object.

The present paper differs from these systems in one principal respect. The distinctive feature is not the use of a DAG per se, but the explicit separation of *order*, *closure maturity*, and *finality* as distinct protocol-level semantic objects.

2.3 Congestion, Delayed Acceptability, and Strategic Deviation

Wu et al. [7] identify the strongest current challenge to the security analysis of high-rate DAG protocols. Their Congestible Blockchain Model shows that propagation completion need not coincide with acceptability once block jam and late-predecessor effects are present. This result is directly relevant here because it shifts the analytical focus from pure relay delay to dependency visibility and processing delay.

The incentive literature is also relevant. Strategic withholding attacks were first articulated clearly in the selfish mining literature [3], where delayed publication changes the effective security margin even without violating PoW validity. Closure manipulation in PHASMDAG is not identical to selfish mining, but it belongs to the same broad family of attacks in which timing asymmetry and selective release alter the consensus-visible state. Any academically complete treatment of a closure-aware protocol must therefore address strategic publication incentives, not merely structural safety.

2.4 The Congestible Blockchain Model

Wu et al. [7] introduce the Congestible Blockchain Model (CBM), which relaxes the Uniform Delay Bounded Model (UDBM) used in prior security proofs.

Definition 2.1 (CBM, informal)**.** Under CBM, each block B experiences a block-specific propagation delay upper bound D_B that may exceed the nominal network delay D. Specifically, D_B depends on:

1. The bandwidth utilisation at the time of B's propagation (Block Jam).

2. The arrival status of B's predecessors at each receiving node (Late Predecessor).

The actual delay for block B at node v is:

$$\Delta_{\text{actual}}(B, v) = \Delta_{\text{prop}}(B, v) + \Delta_{\text{proc}}(B, v) \tag{1}$$

where $\Delta_{\text{proc}}(B, v)$ is the processing delay caused by late predecessors of B that have not yet arrived at v.

Theorem 2.2 (Security degradation under CBM [7])**.** *For a DAG-based PoW protocol under LP attack with s node groups, the chain growth rate of affected blocks is:*

$$g = (1 - \sigma)\,\gamma\, f, \qquad \gamma = \frac{\alpha}{1 + \alpha f\, \mathbb{E}[\Delta]} \tag{2}$$

where σ is the attacker's hash rate fraction, $\alpha = 1 - \sigma$ is the honest fraction, and $\mathbb{E}[\Delta]$ is the expected actual delay. The security threshold $\beta < \gamma$ shrinks as $\mathbb{E}[\Delta]$ increases.

2.5 The Decoupling Assumption

Definition 2.3 (Decoupling Assumption [7])**.** A DAG-based PoW protocol satisfies the Decoupling Assumption if there exists a class of priority blocks such that:

1. Priority blocks propagate to all honest nodes within fixed delay D.
2. Upon receipt, honest nodes can immediately accept priority blocks and build upon them.

Block Jam negates condition (1); Late Predecessor negates condition (2). Zhang and Preneel [8] further show that this assumption is not merely a technical convenience—it is *necessary* for the existing proof structure of GhostDAG-family protocols, because the K-parameter derivation fundamentally requires that the anticone of any block is bounded by K with high probability, which in turn requires that blocks are accepted within D.

3 Problem Statement: The Semantic Conflation

3.1 Three Conflated Semantics

The following conflation appears in GhostDAG-family protocols:

Definition 3.1 (Semantic Conflation)**.** GhostDAG-family protocols use a single structural object—the blue score / K-cluster colouring—to simultaneously represent three distinct semantic roles:

1. **Order**: the canonical position of a block in the DAG's preferred history.
2. **Acceptability**: whether a block's dependency world is sufficiently complete for honest miners to safely build upon it.
3. **Finality**: the economic confidence that a block will not be reorged.

Under UDBM with low f, these three roles approximately coincide. Under CBM with high f, they diverge: a block can be structurally well-ordered (high blue score) yet have an immature dependency world (low acceptability), and acceptability does not guarantee finality.

This conflation is not merely an implementation oversight; it is a *mathematical* property of protocols whose consensus functional is purely structural:

$$\text{GhostDAG:} \quad \max_{S} U_{\text{order}}(S) \qquad \text{(structural concurrency only)} \tag{3}$$

When $U_{\text{order}}(S)$ is the sole objective, there is no mechanism within the protocol to distinguish between "structurally legal but closure-fragile" support and "structurally legal and closure-mature" support.

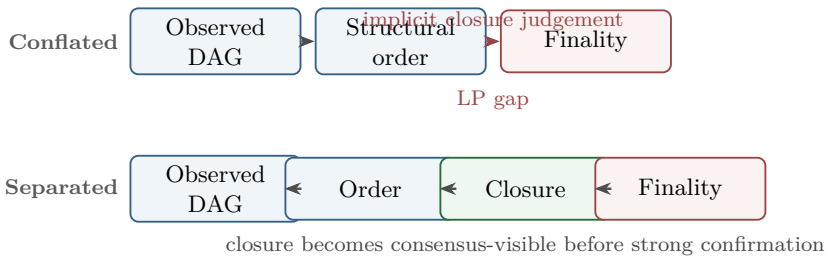

Figure 1: Why an explicit closure layer is needed. Under LP attack, structural ordering can progress before the dependency world is mature enough for strong confirmation. PHASMDAG inserts closure as an explicit protocol stage rather than leaving that judgement to implementation heuristics.

3.2 Why Non-Consensus Mitigations Are Necessary But Insufficient

Several implementation-level mitigations are natural responses to the LP/BJ challenge:

- **Header-First Propagation**: decouple header acceptance from body completion, allowing GhostDAG to progress on header closure alone.
- **Dependency-Aware Relay Priority**: prioritise relay of blocks that unblock the most pending successors.
- **Telemetry**: expose dependency-completion and orphan-age metrics.
- **Adaptive Safety Margins**: locally increase conservatism when network stress is detected.

These mitigations are valuable and should be pursued whenever possible. However, they share a common limitation: they protect a protocol whose core mathematical object remains structurally blind to closure.

Proposition 3.2. *Engineering mitigations that preserve the purely structural consensus objective* $\max U_{\text{order}}(S)$ *can reduce the magnitude of closure fragility but cannot detect or penalise it within the consensus protocol itself. Therefore, the security guarantee remains dependent on the Decoupling Assumption, albeit at a lower effective delay.*

Header-first, for example, narrows the LP attack surface from the body layer to the header layer, but does not eliminate it. The question "is this block's dependency world mature enough for strong finality?" remains unanswerable within the protocol—it is answered only by heuristics in the implementation layer. This semantic mismatch is summarised visually in Figure 1.

3.3 Formal Problem Definition

Definition 3.3 (The PHASMDAG Problem)**.** Design a DAG consensus protocol that:

1. preserves GhostDAG's ability to capture structural concurrency,
2. makes closure maturity a first-class protocol state,
3. derives finality from both order strength and closure maturity,
4. maintains security guarantees under the CBM without relying on the Decoupling Assumption, and
5. degrades gracefully under LP attacks and bandwidth congestion.

4 Protocol Design

4.1 Three Semantic Layers

PHASMDAG reasons over three explicit semantic layers:

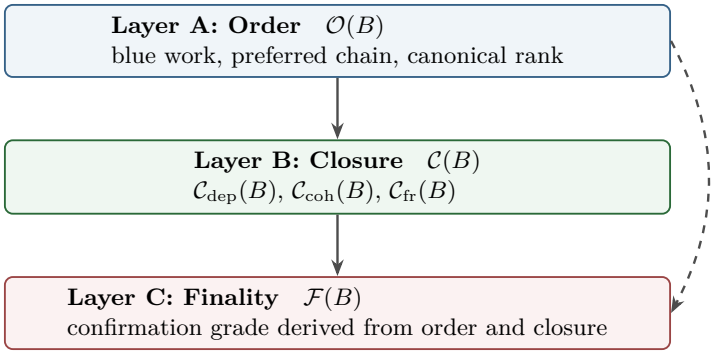

Figure 2: The three semantic layers of PHASMDAG. The Order layer feeds the Closure layer; both jointly determine the Finality layer. The dashed arrow indicates that Order also feeds Finality directly, not only through Closure.

Layer A — Order Layer determines structural support and preferred ordering relations in the DAG. In PHASMDAG-v_0, this layer is fixed to a GhostDAG-style *blue-work lineage*; alternative order cores are future variants, not part of the base protocol.

Layer B — Closure Layer measures whether a block and its dependency world are sufficiently mature, stable, and coherently visible across honest nodes. This is the additional layer absent from GhostDAG-family protocols.

Layer C — Finality Layer derives confirmation/finality from both order and closure, rather than from order alone.

The relationship between layers is illustrated in Figure 2.

4.2 Core Quantities

For each block B in the observed DAG $\mathcal{G} = (V, E)$, two core quantities.

Definition 4.1 (Order Strength)**.** For PHASMDAG-v_0, the *order strength* of block B is the block's position on the GhostDAG blue-work lineage:

$$\mathcal{O}(B) := \text{BlueWork}(B), \qquad \mathcal{O}(B) \in \mathbb{R}_{\geq 0} \tag{4}$$

a structural support quantity measuring how strongly the DAG structure supports placing B in the canonical history. The canonical order rank $r(B)$ is the deterministic rank induced by this blue-work order core.

Definition 4.2 (Relevant Support Window)**.** Let $H \in \mathbb{N}$ be a protocol parameter. The *relevant support window* of block B is

$$\mathcal{W}_H(B) := \{D \in \text{desc}(B) : 0 < r(D) - r(B) \leq H\}. \tag{5}$$

All closure metrics in PHASMDAG-v_0 are computed over $\mathcal{W}_H(B)$ so that they are bounded, deterministic, and auditable.

Definition 4.3 (Closure Sub-Metrics)**.** Let $W(B)$ denote the PoW work attributed to block B. For each $D \in \mathcal{W}_H(B)$, let $\mathbf{1}_{\text{closed}}(D) = 1$ if all protocol-required predecessor objects of D have been observed and validated at the evaluating node, and 0 otherwise. Let $c^\star(B)$ denote the canonical child of B on the blue-work lineage, and let $\pi_B(D)$ denote the first child of B on the path from B

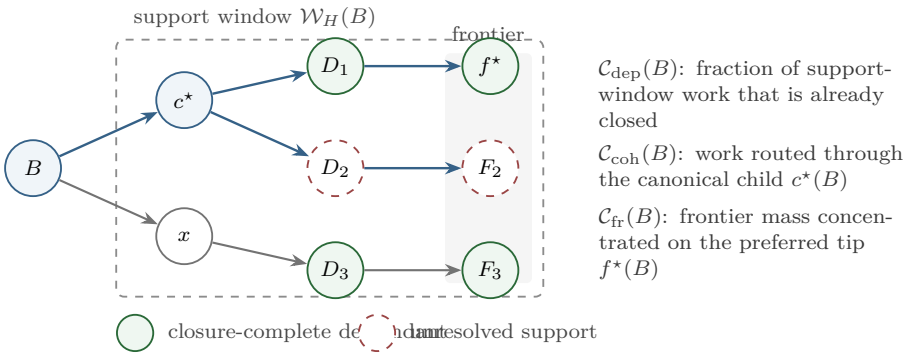

Figure 3: Closure decomposition around block B. The support window $\mathcal{W}_H(B)$ supplies the auditable evidence set; $\mathcal{C}_{\text{dep}}(B)$ measures how much of that work is closure-complete, $\mathcal{C}_{\text{coh}}(B)$ measures how much support flows through the canonical child $c^\star(B)$, and $\mathcal{C}_{\text{fr}}(B)$ measures whether frontier work is concentrated on the preferred tip $f^\star(B)$.

to D. Let $\text{Front}_H(B)$ be the set of maximal blocks in $\mathcal{W}_H(B)$, and let $f^\star(B) \in \text{Front}_H(B)$ be the frontier tip that extends $c^\star(B)$ when such a tip exists. The sub-metrics are defined by:

$$\mathcal{C}_{\text{dep}}(B) = \frac{\sum_{D \in \mathcal{W}_H(B)} W(D)\, \mathbf{1}_{\text{closed}}(D)}{\sum_{D \in \mathcal{W}_H(B)} W(D)} \tag{6}$$

$$\mathcal{C}_{\text{coh}}(B) = \frac{\sum_{D \in \mathcal{W}_H(B)} W(D)\, \mathbf{1}[\pi_B(D) = c^\star(B)]}{\sum_{D \in \mathcal{W}_H(B)} W(D)} \tag{7}$$

$$\mathcal{C}_{\text{fr}}(B) = \frac{W(f^\star(B))}{\sum_{F \in \text{Front}_H(B)} W(F)} \tag{8}$$

with the convention that each ratio is 0 when its denominator is 0.

Definition 4.4 (Closure Level)**.** The *closure level* of block B is the deterministic aggregation of the three closure sub-metrics:

$$\mathcal{C}(B) = g_{v_0}(\mathcal{C}_{\text{dep}}(B), \mathcal{C}_{\text{coh}}(B), \mathcal{C}_{\text{fr}}(B)) \tag{9}$$

where PHASMDAG-v_0 uses the weighted geometric mean

$$g_{v_0}(x, y, z) = x^{\eta_{\text{dep}}} y^{\eta_{\text{coh}}} z^{\eta_{\text{fr}}}, \qquad \eta_{\text{dep}} + \eta_{\text{coh}} + \eta_{\text{fr}} = 1. \tag{10}$$

This aggregation makes a low value in any one dimension visibly reduce the overall closure score.

Remark 4.5. $\mathcal{C}(B)$ is *not* a function of observed propagation delay. It measures structural properties of the dependency world (completeness, stability, coherence), not temporal observations. This is a deliberate design choice: observed delays are node-specific and vulnerable to topology bias [7], whereas structural completeness can, in principle, be determined deterministically from the DAG structure that honest nodes eventually agree on.

Remark 4.6 (Why the decomposition matters). The protocol now exposes three attack surfaces instead of hiding them inside one all-purpose closure scalar: Equation (6) captures dependency completeness, Equation (7) captures descendant-support coherence, and Equation (8) captures frontier stability. Each can be analysed, simulated, and attacked separately.

Figure 3 shows how these three quantities are read off from the local descendant structure around a block.

4.3 Bi-Objective Consensus Functional

The principal departure in PHASMDAG is the replacement of the purely structural objective (3) with a bi-objective functional:

Definition 4.7 (Bi-Objective Consensus Functional)**.** Given a candidate consensus-supporting structure S, the protocol evaluates:

$$\Phi(S) = U_{\text{order}}(S) - \lambda \cdot R_{\text{closure}}(S) \tag{11}$$

where:

- $U_{\text{order}}(S)$ rewards coherent structural order, useful concurrency, descendant-backed support, and stable preferred-chain evolution under the blue-work order core.
- $R_{\text{closure}}(S)$ is a protocol-visible closure-risk functional rather than a prose penalty.
- $\lambda > 0$ is a control parameter setting the tradeoff between structural concurrency capture and closure conservatism.

Definition 4.8 (Closure-Risk Functional)**.** For a candidate set S, define

$$R_{\text{closure}}(S) = a \sum_{B \in S} (1 - \mathcal{C}_{\text{dep}}(B)) + b \sum_{B \in S} (1 - \mathcal{C}_{\text{coh}}(B)) + c \sum_{B \in S} (1 - \mathcal{C}_{\text{fr}}(B)) + d \cdot \text{FragilityCoupling}(S) \tag{12}$$

where $a, b, c, d \geq 0$ are protocol parameters. The first three terms are block-local penalties. The last term captures set-level brittleness: let

$$\mathcal{U}(B) := \{D \in \mathcal{W}_H(B) : \mathbf{1}_{\text{closed}}(D) = 0\}, \tag{13}$$

then

$$\text{FragilityCoupling}(S) := \frac{1}{|P_2(S)|} \sum_{\{B_i, B_j\} \in P_2(S)} \frac{\sum_{D \in \mathcal{U}(B_i) \cap \mathcal{U}(B_j)} W(D)}{\sum_{D \in \mathcal{U}(B_i) \cup \mathcal{U}(B_j)} W(D)}, \tag{14}$$

with value 0 when the denominator is 0. Here $P_2(S)$ is the set of unordered pairs in S.

Remark 4.9 (Auditability of R_{closure}). Equation (12) makes closure risk auditable at two levels: nodes can explain which block-local term depressed a given block's score, and they can separately inspect whether a set of otherwise acceptable blocks shares the same unresolved support pockets through FragilityCoupling(S).

This marks the fundamental departure from GhostDAG: **the protocol no longer maximises structure alone.** It instead maximises a combination of structural utility and closure robustness.

Remark 4.10 (λ semantics). The parameter λ has operational and economic semantics:

- Small λ: the protocol is more optimistic, capturing more raw concurrency but tolerating more closure fragility.
- Large λ: the protocol is more conservative, sacrificing some concurrency for stronger closure guarantees.
- $\lambda \to 0$: PHASMDAG reduces to GhostDAG (pure structural optimisation).

Whether λ is a static protocol constant or an epoch-level consensus-updated parameter is an open design choice discussed in Section 8.3.

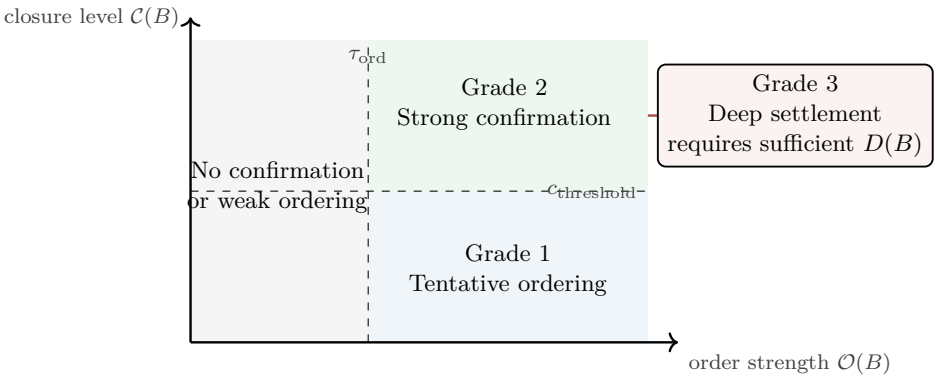

Figure 4: Confirmation-grade geometry. Grade 1 requires sufficiently strong structural ordering, Grade 2 additionally requires closure above the strong confirmation threshold, and Grade 3 adds sufficient settlement depth on top of Grade 2.

4.4 Finality Functional

Definition 4.11 (Closure-Aware Finality)**.** The finality of block B is determined by a joint function:

$$\mathcal{F}(B) = f\big(\mathcal{O}(B),\, \mathcal{C}(B),\, D(B)\big) \tag{15}$$

where $D(B)$ represents settlement depth / descendant accumulation. A simple parameterised form is:

$$\mathcal{F}(B) = \sigma\big(\alpha \cdot \mathcal{O}(B) + \beta \cdot \mathcal{C}(B) + \gamma \cdot D(B) - \tau\big) \tag{16}$$

where σ is a monotone squashing or thresholding function, and $\alpha, \beta, \gamma, \tau$ are protocol parameters.

This means finality is no longer a direct byproduct of blue-work accumulation. A block with high order strength but low closure maturity receives only tentative confirmation; strong confirmation requires both conditions.

4.5 Three Confirmation Grades

The protocol exposes three grades of confirmation:

Grade 1 — Tentative Ordering Block B is structurally well-placed in the DAG, but closure maturity is not yet sufficient for strong trust. Formally: $\mathcal{O}(B)$ exceeds a threshold, but $\mathcal{C}(B)$ is below the strong-confirmation threshold.

Grade 2 — Strong Confirmation Block B has both high structural support and sufficiently high closure maturity. Formally: both $\mathcal{O}(B)$ and $\mathcal{C}(B)$ exceed their respective thresholds.

Grade 3 — Deep Settlement Block B has accumulated enough joint support that rollback becomes economically or combinatorially negligible. Formally: $\mathcal{F}(B)$ exceeds a high-confidence threshold.

This replaces the traditional "one confirmation number fits all" model with semantically differentiated confirmation objects. The geometry of these grades is illustrated in Figure 4.

4.6 Confirmation Invariants

Invariant 4.12 (Tentative Ordering Bounded Churn). There exists a protocol parameter ρ_{tent} such that once a block B attains Grade 1, any later honest view that still places B in the canonical order may change its rank by at most ρ_{tent}. Tentative ordering may move, but only within a bounded local reorder depth.

Invariant 4.13 (Strong Confirmation Agreement). No two honest nodes may assign Grade 2 to conflicting blocks at the same virtual position in the canonical order. Strong confirmation is therefore a protocol object, not merely a UI label.

Invariant 4.14 (Deep Settlement Tail). There exists a protocol target $\varepsilon_{\text{settle}}$ such that any block assigned Grade 3 has rollback probability at most $\varepsilon_{\text{settle}}$ under the stated network and adversarial assumptions.

5 Security Analysis under CBM

5.1 Threat Model

The CBM threat model of [7] is adopted with three layers:

1. **Analytical layer**: an abstract CBM attacker who can partition honest nodes into s groups and selectively delay block delivery.
2. **Realisable layer**: attacks implementable via BGP hijacking, Eclipse attacks, or strategic peer selection.
3. **Implementation layer**: representative node behaviour under stress, including orphan-pool dynamics and header-body skew.

5.2 Chain Growth under Closure Awareness

This subsection is intentionally preliminary. It does *not* claim a completed security proof for PHASMDAG. Instead, it records one concrete modelling ansatz for how closure awareness may change the LP security envelope, assuming the deterministic v_0 closure metric and eventual convergence of its sub-metrics.

Lemma 5.1 (Preliminary closure-modulated chain-growth heuristic)**.** *Under CBM with LP attack parameter s, let $\mathcal{C}_{\min}(t)$ denote the minimum closure level among blocks that contribute to the honest chain's structural support at time t. Under the heuristic assumption that unresolved closure mass linearly modulates delay exposure, the effective chain growth rate is:*

$$g_{\text{PHASM}} = (1-\sigma)\,\gamma_{\text{eff}}\,f \tag{17}$$

where:

$$\gamma_{\text{eff}} = \frac{\alpha \cdot \mathcal{C}_{\min}}{1 + \alpha f \cdot \mathbb{E}[\Delta] \cdot (1 - \mathcal{C}_{\min})} \tag{18}$$

Proof sketch. Under GhostDAG (Definition 2.2), the chain growth rate is $g = (1-\sigma)\gamma f$ where $\gamma = \alpha/(1+\alpha f \mathbb{E}[\Delta])$. The key quantity is $\mathbb{E}[\Delta]$, the expected actual delay including processing delay from late predecessors.

In PHASMDAG, the bi-objective functional $\Phi = U_{\text{order}} - \lambda R_{\text{closure}}$ weights each block's contribution to structural support by its closure maturity. Specifically, a block B with closure level $\mathcal{C}(B)$ contributes an effective weight of $w(B) = \mathcal{C}(B)$ to the order utility; the remaining fraction $(1-\mathcal{C}(B))$ is absorbed by the closure risk penalty.

This means that only the *resolved* portion of the dependency world contributes to effective chain growth. The unresolved fraction $(1 - \mathcal{C}_{\min})$ carries the delay risk. Substituting $\mathbb{E}[\Delta] \cdot (1 - \mathcal{C}_{\min})$ for $\mathbb{E}[\Delta]$ in Definition 2.2, and noting that the numerator's honest hash power is modulated by $\mathcal{C}_{\min}$ (since only closure-mature blocks contribute full weight), we obtain:

$$\gamma_{\text{eff}} = \frac{\alpha \cdot \mathcal{C}_{\min}}{1 + \alpha f \cdot \mathbb{E}[\Delta] \cdot (1 - \mathcal{C}_{\min})}$$

When $\mathcal{C}_{\min} = 1$ (full closure), this reduces to $\gamma_{\text{eff}} = \alpha/(1+0) = \alpha$, which is the GhostDAG rate under UDBM with zero LP effect—consistent with the intuition that full closure maturity eliminates delay exposure. When $\mathcal{C}_{\min} = 0$ (complete closure failure), $\gamma_{\text{eff}} = 0$, reflecting that no effective chain growth is possible when no block's dependency world is mature. This derivation should be read as a candidate parameterisation, not as a finished reduction from the CBM theorem of Wu et al. □

Proposition 5.2 (Preliminary dominance region)**.** *Under LP attack, PHASMDAG's effective security threshold* γ_{eff} *dominates GhostDAG's* γ *within the parameter region*

$$\mathcal{C}_{\min} \geq \frac{1}{1 + 2\alpha f \mathbb{E}[\Delta]}. \tag{19}$$

Within this region, the heuristic ansatz of Definition 5.1 implies $\gamma_{\text{eff}} \geq \gamma$*.*

Proof sketch. Compare γ from Definition 2.2 with γ_{eff} from Definition 5.1:

$$\gamma = \frac{\alpha}{1 + \alpha f \cdot \mathbb{E}[\Delta]} \tag{20}$$

$$\gamma_{\text{eff}} = \frac{\alpha \cdot \mathcal{C}_{\min}}{1 + \alpha f \cdot \mathbb{E}[\Delta] \cdot (1 - \mathcal{C}_{\min})} \tag{21}$$

Cross-multiplying the claim $\gamma_{\text{eff}} \geq \gamma$ gives

$$\alpha \cdot \mathcal{C}_{\min} \cdot (1 + \alpha f \mathbb{E}[\Delta]) \geq \alpha \cdot (1 + \alpha f \mathbb{E}[\Delta](1 - \mathcal{C}_{\min}))$$

Simplify (both sides have $\alpha > 0$):

$$\mathcal{C}_{\min} + \alpha f \mathbb{E}[\Delta] \cdot \mathcal{C}_{\min} \geq 1 + \alpha f \mathbb{E}[\Delta] - \alpha f \mathbb{E}[\Delta] \cdot \mathcal{C}_{\min}$$

Rearranging:

$$2\alpha f \mathbb{E}[\Delta] \cdot \mathcal{C}_{\min} + \mathcal{C}_{\min} - 1 \geq 0$$

This holds when $\mathcal{C}_{\min}$ is sufficiently large relative to the LP effect. Specifically, the inequality holds when $\mathcal{C}_{\min} \geq 1/(1 + 2\alpha f \mathbb{E}[\Delta])$. Since the right-hand side is always < 1, there exists a non-trivial region where the heuristic threshold of PHASMDAG exceeds GhostDAG's. □

Remark 5.3 (Interpretation cost of closure awareness). Definition 5.2 does *not* establish a universal improvement result. It identifies a preliminary dominance region under the specific heuristic ansatz of Definition 5.1. The cost is that blocks with $\mathcal{C} < 1$ receive weaker finality (Definition 4.11), which means the protocol may delay strong confirmation for blocks whose dependency world is immature. This behaviour is methodologically preferable: strong confirmation should be delayed rather than granted on the basis of a structural score that ignores closure fragility.

5.3 Comparison with GhostDAG under LP Attack

Table 1 summarises the key differences. The essential observation is that PHASMDAG *absorbs* the LP effect into the protocol's mathematical core rather than treating it as an external implementation concern.

Table 1: Preliminary security behaviour comparison under LP attack

Property	GhostDAG	PHASMDAG
Chain growth rate	$g = (1-\sigma)\gamma f$	$g = (1-\sigma)\gamma_{\text{eff}} f$
Security threshold	$\beta < \gamma$	candidate $\beta < \gamma_{\text{eff}}$
Delay exposure	$\mathbb{E}[\Delta]$ (unmodulated)	$\mathbb{E}[\Delta] \cdot (1-\mathcal{C}_{\min})$
Closure fragility	Invisible to protocol	First-class state $\mathcal{C}(B)$
Strong finality basis	$\mathcal{O}(B)$ alone	$\mathcal{O}(B) \wedge \mathcal{C}(B)$

5.4 Quality under Heterogeneous Visibility

Under LP attack, different honest nodes may have different views of the DAG. GhostDAG's security proofs assume that honest nodes converge to consistent blue-work orderings; under CBM, this convergence may be delayed or incomplete.

Conjecture 5.4 (Closure coherence theorem target)**.** *Under CBM with honest majority, if $\mathcal{C}(B) \geq c_{\text{threshold}}$ at some honest node v, then within bounded additional time $\Delta_{\text{coherence}}$, all honest nodes will also observe $\mathcal{C}(B) \geq c_{\text{threshold}} - \epsilon$ for some small $\epsilon > 0$.*

This conjecture, if proven, would establish that closure level converges across honest nodes—a prerequisite for closure-aware finality to be consistent. It is the *central theorem target* of PHASMDAG: without sufficient convergence of $\mathcal{C}_{\text{dep}}$, $\mathcal{C}_{\text{coh}}$, and $\mathcal{C}_{\text{fr}}$, Grade 2 cannot be a consensus-meaningful object. Its proof or disproof is therefore a primary goal of the formal modelling roadmap (Section 6).

5.5 Incentive Compatibility and Closure Manipulation

Introducing $\mathcal{C}(B)$ and $R_{\text{closure}}(S)$ as first-class protocol objects creates a new strategic attack surface that does not exist in GhostDAG. In GhostDAG, the only rational miner behaviour is to extend the heaviest blue-work chain; there is no protocol-internal incentive to manipulate structural scores. In PHASMDAG, miners may profit from manipulating closure evidence. This draft therefore does *not* claim that honest, immediate broadcasting is already a Nash equilibrium under PHASMDAG.

Closure suppression. A miner could withhold a predecessor block B^* to deliberately depress $\mathcal{C}(B)$ for a rival's blocks that reference B^* transitively. This delays the rival's strong confirmation and may shift the preferred-chain tip toward the attacker's own blocks, since R_{closure} penalises the rival's dependency fragility.

Closure inflation. Conversely, a miner could strategically release closure evidence (by broadcasting withheld predecessors) at a time chosen to maximise the boost to their own blocks' $\mathcal{C}$, potentially timing the release to outcompete a rival's chain tip just before a difficulty retarget.

Frontier manipulation. Since $\mathcal{C}(B)$ depends on descendant support and frontier concentration (Equations (7) and (8)), a miner could mine descendant blocks that strategically include or exclude closure-confirming references, steering the frontier to inflate or suppress $\mathcal{C}$ for selected blocks.

These threats are not theoretical curiosities—they are the direct consequence of making closure a consensus-relevant quantity. The full game-theoretic treatment is left for future work, but the present paper identifies the following as *required incentive-compatibility goals* for any viable PHASMDAG specification:

1. **Non-profitability of suppression**: withholding predecessors should not yield a net revenue advantage, accounting for the attacker's own lost block rewards and the cost of maintaining the suppression.
2. **Convergence under strategic release**: even if miners strategically time the release of closure evidence, $\mathcal{C}(B)$ should converge to the same value it would have reached under honest immediate broadcasting, within bounded additional time.
3. **Sybil resistance**: cheap creation of low-work descendant blocks must not inflate $\mathcal{C}(B)$; the work-weighted definitions in Equations (6) to (8) are intended to enforce this.

Remark 5.5 (Relationship to selfish mining). Closure manipulation is structurally similar to selfish mining [8]: both exploit information asymmetry and timing control. However, closure manipulation operates on a different axis—it manipulates *dependency maturity* rather than *structural weight*—and may interact non-trivially with selfish mining strategies. A complete analysis of the joint strategy space is an important direction for future work.

6 Formal Modelling Approach

This section develops a compositional formal model aligned with the three semantic layers.

6.1 Compositional State Machine

Each block B maintains three logical state components:

1. **Structural state** $\Sigma_{\text{ord}}(B)$: parent set, selected-parent relation, blue-work score, ordering rank. This is essentially the existing GhostDAG state.
2. **Closure state** $\Sigma_{\text{cls}}(B)$: relevant support window $\mathcal{W}_H(B)$, the three sub-metrics $\mathcal{C}_{\text{dep}}(B)$, $\mathcal{C}_{\text{coh}}(B)$, $\mathcal{C}_{\text{fr}}(B)$, unresolved-support set $\mathcal{U}(B)$, and aggregate closure level $\mathcal{C}(B)$.
3. **Finality state** $\Sigma_{\text{fin}}(B)$: tentative inclusion status, strong confirmation status, settlement confidence $\mathcal{F}(B)$.

The full state machine is the product $\Sigma(B) = \Sigma_{\text{ord}}(B) \times \Sigma_{\text{cls}}(B) \times \Sigma_{\text{fin}}(B)$, with admissible transitions defined by the protocol rules. The transition order is strict: Σ_{ord} is computed first (since closure depends on structural state), then Σ_{cls}, then Σ_{fin}.

6.2 Local Views and Disagreement Metrics

The key modelling difficulty is that PHASMDAG is evaluated under partial and heterogeneous visibility. The protocol therefore requires an explicit notation for node-local views and for cross-node disagreement in closure scores.

Definition 6.1 (Node-Local DAG View)**.** Let $\mathcal{G}_v(t)$ denote the accepted header-DAG observed by honest node v at time t. All protocol quantities are node-local during execution:

$$\mathcal{O}_v(B,t), \qquad \mathcal{C}_v(B,t), \qquad \mathcal{F}_v(B,t), \qquad \Phi_v(S,t).$$

Consensus safety requires that these local quantities converge sufficiently fast after network stabilisation.

Definition 6.2 (Closure Disagreement)**.** For any block B visible to the honest node set H_t at time t, define the honest-node closure disagreement by

$$\Delta_{\mathcal{C}}(B,t) := \max_{u,v \in H_t} \left| \mathcal{C}_u(B,t) - \mathcal{C}_v(B,t) \right|. \tag{22}$$

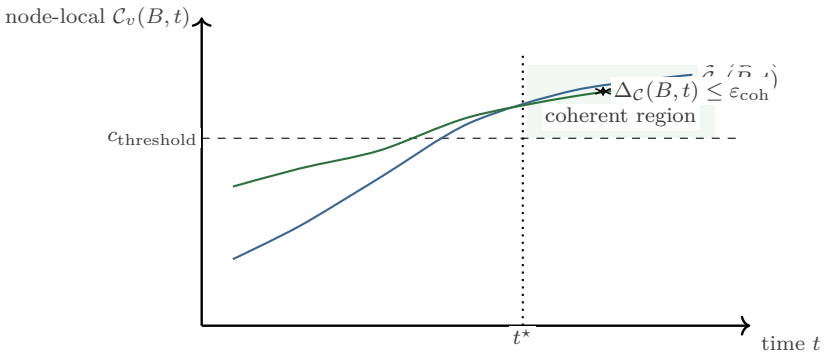

Figure 5: Closure coherence as a convergence target. Strong confirmation should only become globally meaningful once node-local closure estimates are both above $c_{\text{threshold}}$ and sufficiently close to one another.

The central modelling objective is to show that $\Delta_{\mathcal{C}}(B,t)$ becomes small quickly enough to support consistent Grade 2 decisions.

Definition 6.3 (Closure-Coherent Strong Confirmation Target)**.** A block B is said to satisfy the *closure-coherent strong-confirmation target* at time t if

$$\min_{v \in H_t} \mathcal{C}_v(B,t) \ge c_{\text{threshold}} \qquad \text{and} \qquad \Delta_{\mathcal{C}}(B,t) \le \varepsilon_{\text{coh}} \tag{23}$$

for a protocol tolerance $\varepsilon_{\text{coh}} > 0$. Definition 5.4 may then be read as a theorem target asserting eventual attainment of (23) for structurally supported honest blocks.

This convergence target is visualised in Figure 5.

6.3 Block Processing Algorithm

Algorithm 1 specifies the high-level block processing pipeline for PHASMDAG, making the three-layer computation order explicit.

6.4 Closure Level Computation

The most critical modelling challenge is the computation of $\mathcal{C}(B)$. PHASMDAG now treats this as a family of explicitly separated design choices, with one standardised instantiation and one experimental variant.

Approach A: Deterministic attestation-based closure (v_0). Approach A is the **only** standardised closure computation in PHASMDAG-v_0. Its simplest descendant-attestation form is the PoW-weighted statistic

$$\mathcal{C}_A(B) = \frac{\sum_{B' \in \text{desc}(B),\, \mathbf{1}_{\text{closed}}(B')=1} W(B')}{\sum_{B' \in \text{desc}(B)} W(B')}. \tag{24}$$

Here $W(B')$ denotes the PoW weight, accumulated work, or difficulty-equivalent contribution of descendant block B'. This makes closure evidence an economic-security quantity rather than a raw descendant count, and prevents a minority miner from cheaply inflating closure with low-value descendants alone. It instantiates $\mathcal{C}_{\text{dep}}(B)$, $\mathcal{C}_{\text{coh}}(B)$, and $\mathcal{C}_{\text{fr}}(B)$ exactly as in Equations (6) to (8) and aggregates them with Equation (10). Equation (24) is the unwindowed intuition; PHASMDAG-v_0 implements the same idea through the bounded support-window version $\mathcal{C}_{\text{dep}}(B)$ in Equation (6).

Table 2: Closure computation strategies

Property	Approach A	Approach B
Deterministic across nodes	Strong	Weak–Medium
Responsiveness	Lower	Higher
Formal proof friendliness	High	Low
Topology bias risk	Low	Higher
Implementation complexity	Lower	Higher

Table 3: Protocol parameter space

Parameter	Type	Constraints
λ	Protocol-level	v_0: static consensus constant; dynamic adjustment excluded from initial model (Section 8.3)
H	Closure window	Support-horizon size for $\mathcal{W}_H(B)$; must be large enough to capture relevant descendant evidence
$\eta_{\text{dep}}, \eta_{\text{coh}}, \eta_{\text{fr}}$	Aggregation	Non-negative and sum to 1 in Equation (10)
a, b, c, d	Closure-risk weights	Non-negative; determine auditably how $R_{\text{closure}}(S)$ trades block-local vs. set-level fragility
$\alpha, \beta, \gamma, \tau$	Finality function	Must ensure monotonicity of $\mathcal{F}$ in both $\mathcal{O}$ and $\mathcal{C}$
$c_{\text{threshold}}$	Closure	Minimum $\mathcal{C}$ for strong confirmation
ρ_{tent}	Tentative ordering	Maximum permitted rank churn for Grade 1 (Definition 4.12)
$\varepsilon_{\text{settle}}$	Settlement target	Maximum rollback tail for Grade 3 (Definition 4.14)
$\mathcal{C}$ definition	Protocol design	v_0: Approach A only; Approach B is experimental (Definition 6.4)

It uses only structural evidence observable in the accepted header-DAG plus the validation state required for $\mathbf{1}_{\text{closed}}$. This makes the metric deterministic, decomposable, and auditable.

Approach B: Probabilistic delay-posterior closure. Define $\mathcal{C}(B)$ as the probability that B's full dependency world has been received by a sufficient fraction of honest nodes:

$$\mathcal{C}_B(B) = \Pr\Big[\text{fraction of honest nodes that have accepted all preds of } B \geq \theta \;\Big|\; \text{observed evidence}\Big] \tag{25}$$

This is more expressive—it can incorporate timing evidence and network models—but harder to compute consistently across nodes and harder to prove secure.

Remark 6.4 (Version policy for closure computation). Approach A is the reference instantiation for PHASMDAG-v_0. Approach B remains explicitly experimental. This is a versioning decision, not merely a modelling preference: the base protocol optimises for determinism and proof-friendliness first.

6.5 Parameter Space

The protocol introduces the following parameters that must be analysed formally:

Table 4: Evaluation metrics

Metric	Description
Chain growth rate g	Blocks per second entering the canonical order
Heuristic effective security threshold γ_{eff}	Candidate minimum honest fraction for security under LP
Realised TPS	Actual transaction throughput under imperfect networks
Tentative confirmation latency	Time from block creation to Grade 1
Strong confirmation latency	Time from block creation to Grade 2
Deep settlement latency	Time from block creation to Grade 3
Closure convergence time	Time for $\mathcal{C}(B)$ to reach $c_{\text{threshold}}$ across honest nodes
P95/P99 finality tail	Tail behaviour of confirmation latency under stress

6.6 Formal Verification Targets

The following properties are targets for formal verification:

1. **Tentative bounded churn**: prove or bound Definition 4.12 under the blue-work order core.
2. **Strong-confirmation safety**: prove Definition 4.13, namely that no two honest nodes assign conflicting Grade 2 confirmations at the same virtual position.
3. **Settlement tail**: prove or upper-bound Definition 4.14.
4. **Liveness**: every honestly mined block eventually reaches tentative ordering (Grade 1) if the network eventually stabilises.
5. **Closure coherence**: establish or refute Definition 5.4, which is the central theorem target of the protocol.
6. **Dominance-region validation**: determine whether the heuristic region in Definition 5.2 survives a full CBM proof or only a simulation-level approximation.

7 Engineering Verification

7.1 Simulation Methodology

The simulation programme is based on an extension of the SimBlock [11] framework to model CBM conditions:

1. **Network model**: replace the uniform delay model with CBM, incorporating bandwidth limits and per-block propagation delays.
2. **LP attack simulation**: implement the s-group partition attack from [7], allowing the attacker to selectively delay block delivery to specific node groups.
3. **Bandwidth congestion**: simulate Block Jam by enforcing a global bandwidth cap and measuring propagation delays as a function of utilisation.
4. **Closure tracking**: augment each simulated node with the v_0 closure computation (Approach A) and track $\mathcal{C}(B)$ over time.

7.2 Evaluation Metrics

The evaluation is organised around the following metrics, following the common-metrics framework of [8]:

7.3 Comparative Evaluation Plan

1. **Baseline**: GhostDAG under UDBM (optimistic reference).
2. **GhostDAG under CBM**: GhostDAG with LP attack and bandwidth congestion, no mitigations.
3. **GhostDAG + engineering mitigations**: GhostDAG with header-first, dependency-aware relay, and telemetry.
4. **PHASMDAG under CBM**: PHASMDAG with $\mathcal{C}$ computation (Approach A) and bi-objective functional, under the same CBM conditions.

The comparison between configurations (3) and (4) directly tests Definition 3.2: does the protocol-level closure awareness of PHASMDAG provide security benefits beyond what engineering mitigations can achieve?

7.4 Complexity and Incremental Computation

The introduction of Σ_{cls} and Σ_{fin} state machines, frontier stability tracking, and closure attestation caches raises legitimate concerns about computational overhead. A protocol that is theoretically sound but practically unrunnable serves no one. Incremental computation is therefore stipulated as a strict prerequisite for v_0, not an afterthought.

The critical metrics that must be evaluated and bounded before a PHASMDAG prototype is considered viable are:

1. **Amortised per-block update cost for $\mathcal{C}(B)$**: Computing $\mathcal{C}(B)$ for a new block requires updating the three sub-metrics over the affected support windows. Under Approach A (Equations (6) to (8)), this can be formulated incrementally: when a new block B_{new} arrives, only its direct work contribution and its canonical-child / frontier relations need to be propagated to affected ancestors. The target is $O(|\text{parents}|)$ per block, not $O(|\text{ancestors}|)$.
2. **Memory footprint of closure-state indices and attestation caches**: Each block must store: (a) the cumulative work mass needed for $\mathcal{C}_{\text{dep}}(B)$, (b) support-window routing metadata needed for $\mathcal{C}_{\text{coh}}(B)$, and (c) frontier concentration metadata needed for $\mathcal{C}_{\text{fr}}(B)$. These are all bounded by the finite support window H and can be encoded as compact additive counters plus small frontier indices rather than as a full descendant list.
3. **Incremental recomputation depth**: When $\mathcal{C}(B)$ changes (because a new descendant arrived or a predecessor was resolved), closure scores must be updated along the *affected descendant frontier*—not the entire DAG. The expected frontier size is bounded by the merge depth and the number of recent blocks, both of which are $O(K)$ in the current GhostDAG implementation.
4. **Lazy recomputation for deep historical blocks**: Blocks deeper than the finality depth can be considered settled; their $\mathcal{C}$ values are no longer consensus-relevant and need not be recomputed. Pruning strategies can treat $\mathcal{C}$ computation similarly to how existing GhostDAG implementations prune anticone data beyond the pruning depth.

Remark 7.1 (Computational feasibility of Approach A). Approach A is specifically designed for incremental computation: the work-weighted numerators and denominators in Equations (6) to (8) are additive or frontier-local quantities that can be updated without re-walking the entire descendant cone. This contrasts with Approach B (Equation (25)), which would require Bayesian updates over the entire ancestor set and therefore lacks a similarly clean incremental path. This is an additional reason to prefer Approach A for v_0.

Table 5: Phased verification roadmap

Phase	Timeline	Deliverables
1	0–6 mo	Formal model: define $\mathcal{O}$, $\mathcal{C}_{\text{dep}}$, $\mathcal{C}_{\text{coh}}$, $\mathcal{C}_{\text{fr}}$, $\mathcal{C}$, and $\mathcal{F}$; specify state machine and admissible transitions
2	6–12 mo	Simulation: implement CBM/LP scenarios in SimBlock; compare GhostDAG vs PHASMDAG under identical conditions
3	12–18 mo	Security analysis: chain growth under joint order/closure semantics; quality under heterogeneous visibility; adversarial threshold as function of closure degradation
4	18–24 mo	Prototype: implement order layer + closure tracking; expose three confirmation grades; evaluate on a controlled experimental network

Table 6: Delay-weighted colouring vs. PHASMDAG R_{closure}

Dimension	Delay-Weighted	PHASMDAG R_{closure}
Penalisation target	Observed delay	Dependency fragility
Topology bias risk	High	Low
Conceptual cleanliness	Temporal signal → consensus weight	Structural robustness → consensus function
Node consistency	Different nodes see different delays	Structural properties converge

7.5 Phased Verification Roadmap

Note that Phases 1–2 can proceed in parallel with the engineering mitigations (header-first, telemetry, dependency-aware relay) and other implementation-level studies of GhostDAG. The CBM-GhostDAG security proof may be read as a decision gate: if it indicates that engineering mitigations suffice at the target BPS, PHASMDAG may remain chiefly a research direction; if not, it may warrant treatment as a more substantial redesign path.

8 Discussion

8.1 Comparison with Delay-Weighted Colouring

An alternative approach to handling LP effects is delay-weighted colouring—promoting GhostDAG's binary blue/red colouring to a continuous weight based on observed propagation delay. For a consensus-visible v_0 design, PHASMDAG's R_{closure} is the more analytically coherent choice:

The key insight is that R_{closure} penalises *structural fragility* (dependency incompleteness, frontier instability) rather than *temporal observations* (how long a block took to arrive). Since structural properties of the DAG eventually converge across honest nodes, R_{closure} is less vulnerable to the topology bias identified in [7, 8].

8.2 Failure Modes and Design Risks

The proposal is best read alongside an explicit account of its likely failure modes. The most important of these are:

- **Closure metric too conservative**: $\mathcal{C}_{\text{dep}}$, $\mathcal{C}_{\text{coh}}$, or $\mathcal{C}_{\text{fr}}$ may converge too slowly, causing avoidable TPS and finality regressions.

- **Closure metric too weak**: an under-specified sub-metric may merely rename the LP attack surface instead of eliminating it.
- λ **badly chosen**: too small and the protocol collapses back toward GhostDAG-style optimism; too large and it becomes quasi-serial and over-conservative.
- **Set-level fragility under-modelled**: if FragilityCoupling(S) is too weak, individually acceptable blocks may still form a collectively brittle support structure.
- **Closure coherence slower than assumed**: if the sub-metrics do not converge quickly enough across honest nodes, Grade 2 cannot be treated as a safe consensus object.
- **Semantic interface complexity**: the three-grade finality model may be theoretically well motivated yet difficult to expose through higher-layer interfaces without misinterpretation.

8.3 Open Problems

$\mathcal{C}(B)$ **semantics.** The decomposition of $\mathcal{C}(B)$ is now explicit, but the central open problem is no longer "what are the parts?" but "do the parts converge enough across honest nodes?" Approach A (Definition 6.4) gives deterministic rules for $\mathcal{C}_{\text{dep}}$, $\mathcal{C}_{\text{coh}}$, and $\mathcal{C}_{\text{fr}}$, yet the theorem target remains Definition 5.4: proving that these sub-metrics stabilise quickly enough to support consistent Grade 2 confirmation. That is the hardest unresolved problem in the paper.

λ **consensus consistency.** For v_0, λ is a **static protocol constant**, fixed at genesis or updated only via hard fork. This is the simplest design that guarantees consensus consistency: all nodes trivially agree on λ.

However, the optimal λ varies with network conditions: small λ is optimal under light load, large λ under congestion. A dynamic λ that adapts to observed network stress—analogous to a control-theoretic PID controller—is a conceptually interesting but consensus-sensitive extension. The risk is total consensus failure if nodes have slightly misaligned observation windows or if the congestion signal itself is manipulated by an attacker.

Dynamic λ is therefore framed as a *control-theoretic extension candidate*, excluded from the initial formal model due to:

1. Consensus-consistency risk: nodes observing different network conditions would compute different λ, leading to divergent chain tips.
2. Signal manipulation vectors: an attacker could manipulate the congestion signal (e.g., via bandwidth throttling) to trick honest nodes into an unfavourable λ.

Nodes *could* compute a local, non-consensus advisory metric $\lambda_{\text{estimate}}$ strictly for simulation, operator dashboards, and telemetry. This metric must not bear upon block validity or consensus state; it is purely informational. A potential middle ground for future work is epoch-level consensus updates, similar to difficulty adjustment in Bitcoin, but this requires careful analysis of the update rule's attack surface.

Fork distance from GhostDAG. PHASMDAG should perhaps not be understood as a minor variant of GhostDAG, but rather as a more substantial protocol divergence. If ever pursued in earnest, it would probably require a distinct protocol identity (for example, *ClosureDAG*) rather than a simple suffix such as GhostDAG+. This bears directly upon migration, client adaptation, and ecosystem compatibility.

Three-grade confirmation interpretation cost. Moving from a single confirmation scalar to multiple semantic grades introduces an interface problem as well as a protocol problem. Any future

deployment would require a careful theory of how higher layers should interpret tentative ordering, strong confirmation, and deep settlement without collapsing them back into an opaque scalar.

8.4 Protocol-Level versus Implementation-Level Mitigation

Implementation-level mitigations remain necessary, but they are not sufficient. They reduce stress; they do not close the consensus-level semantic gap by themselves. The relevant boundary is:

- **Implementation-level mitigations can reduce** propagation skew, predecessor starvation, and operator blindness.
- **Implementation-level mitigations cannot eliminate** the hidden gap between header progress and strong finality while closure fragility remains non-consensus-visible.
- **Implementation-level mitigations cannot natively penalise** support structures that are individually legal but collectively closure-fragile, because that requires a protocol term like $R_{\text{closure}}(S)$.
- **Implementation-level mitigations cannot provide** a protocol-native semantics for "mature enough to trust" without an object like $\mathcal{C}(B)$ entering the consensus mathematics.

PHASMDAG still depends on such mitigations as infrastructure:

1. **Telemetry** provides the observational basis for $\mathcal{C}(B)$ computation (dependency completion latency, orphan age distributions).
2. **Header-First Propagation** enables the closure layer to operate at the header level independently of body arrival, reducing the LP attack surface that the closure layer must handle.
3. **Dependency-Aware Relay** improves the convergence rate of $\mathcal{C}(B)$ by ensuring that blocks blocking the most pending successors are relayed first.

The more appropriate framing is therefore: *implementation-level mitigations are foundational, but they are not a substitute for protocol redesign; PHASMDAG is the protocol-level completion that makes closure awareness explicit in the consensus mathematics.*

9 Conclusion

This paper has set out a preliminary formulation of PHASMDAG, a bi-objective, closure-aware DAG consensus protocol motivated by the security questions raised by Wu, Wei, Zhang, and Jiang [7].

The central contention has been that the root limitation of GhostDAG-family protocols may lie not merely in the attackability of relay paths, but also in the manner in which one structural object is made to stand simultaneously for ordering, acceptability, and finality. The proposed response is to separate structural order, closure maturity, and finality into distinct mathematical roles, and to reconnect them through an explicit bi-objective consensus functional:

$$\Phi(S) = U_{\text{order}}(S) - \lambda\, R_{\text{closure}}(S)$$

Taken together, the present formulation supports the following tentative claims:

1. Block Jam and Late-Predecessor effects can be brought within the protocol's mathematical core rather than treated solely as external implementation concerns.
2. A concrete and auditable v_0 instantiation of closure can be given through $\mathcal{C}_{\text{dep}}$, $\mathcal{C}_{\text{coh}}$, $\mathcal{C}_{\text{fr}}$, and $R_{\text{closure}}(S)$.
3. A preliminary dominance-region heuristic under LP attack can be stated without thereby claiming a completed universal superiority result (Definition 5.2).

4. Semantically explicit confirmation grades can be coupled to explicit invariants rather than collapsed into a single opaque scalar.
5. Under the modelling choices adopted here, degradation under congestion may be rendered more gradual than an abrupt security collapse.

The principal unresolved difficulty remains closure coherence: whether the deterministic closure sub-metrics converge sufficiently across honest nodes to make Grade 2 globally meaningful. Approach A has been standardised as the reference v_0 design, while Approach B remains an experimental extension. The paper has also set out a phased programme of formal modelling and verification.

PHASMDAG should therefore be read not as an incremental amendment to GhostDAG, but as a more substantial re-specification of what a DAG consensus protocol is taken to optimise. Whether it ultimately warrants treatment as an independent protocol family must depend upon the outcome of the formal and experimental programme set out above.

Acknowledgements

The author thanks colleagues and interlocutors whose discussions on DAG consensus, delayed acceptability, and finality semantics helped sharpen the questions addressed in this paper. Any remaining errors, omissions, or overstatements are the sole responsibility of the author.

References

[1] Y. Sompolinsky and A. Zohar, "Secure High-Rate Transaction Processing in Bitcoin," *IACR Cryptology ePrint Archive*, Report 2013/881, 2013.

[2] J. Garay, A. Kiayias, and N. Leonardos, "The Bitcoin Backbone Protocol: Analysis and Applications," *Advances in Cryptology – EUROCRYPT*, 2015.

[3] I. Eyal and E. G. Sirer, "Majority Is Not Enough: Bitcoin Mining Is Vulnerable," *arXiv preprint arXiv:1311.0243*, 2013.

[4] Y. Sompolinsky and A. Zohar, "SPECTRE: A Fast and Scalable Cryptocurrency Protocol," *IACR Cryptology ePrint Archive*, Report 2016/1159, 2016.

[5] Y. Sompolinsky and A. Zohar, "PHANTOM: A Scalable BlockDAG Protocol," *IACR Cryptology ePrint Archive*, Report 2018/104, 2018.

[6] Y. Sompolinsky, S. Wyborski, and A. Zohar, "PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus," *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT)*, pp. 57–70, 2021.

[7] S. Wu, P. Wei, R. Zhang, and B. Jiang, "Security-Performance Tradeoff in DAG-based Proof-of-Work Blockchain Protocols," *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.

[8] R. Zhang and B. Preneel, "Lay Down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols' Security," *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, pp. 1441–1458, 2019.

[9] H. Yu, "OHIE: Blockchain Scaling Made Simple," *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pp. 908–925, 2021.

[10] V. Bagaria, S. Kannan, D. Tse, G. Fanti, and P. Viswanath, "Prism: Deconstructing the Blockchain to Approach Physical Limits," *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 585–602, 2019.

[11] Y. Aoki, K. Shigeta, and R. Banno, "SimBlock: A Validated Blockchain Simulator," *Proceedings of the IEEE International Conference on Blockchain (Blockchain)*, pp. 131–139, 2019.

Algorithm 1 PHASMDAG Block Processing

Require: Incoming block B_{new} with parent set P
Ensure: Updated DAG state Σ

// — **Layer A: Order** —
Validate header in isolation (PoW, timestamp, format) ▷ Same as GhostDAG
Check all parents exist in Σ_{ord} ▷ `check_parents_exist`
Compute $\mathcal{O}(B_{\text{new}})$ and $r(B_{\text{new}})$ via blue-work ordering
Select preferred parent, compute mergeset, colour blue/red
Update $\Sigma_{\text{ord}}(B_{\text{new}})$

// — **Layer B: Closure** —
Identify affected ancestors A whose support windows include B_{new}
for each affected block A **do**
    Update $\mathcal{W}_H(A)$ and $\mathcal{U}(A)$
    Recompute $\mathcal{C}_{\text{dep}}(A)$, $\mathcal{C}_{\text{coh}}(A)$, and $\mathcal{C}_{\text{fr}}(A)$ ▷ Approach A / v_0
    $\mathcal{C}(A) \leftarrow g_{v_0}(\mathcal{C}_{\text{dep}}(A), \mathcal{C}_{\text{coh}}(A), \mathcal{C}_{\text{fr}}(A))$
end for
Initialise $\Sigma_{\text{cls}}(B_{\text{new}})$

// — **Layer C: Finality** —
Compute $\mathcal{F}(B) = f(\mathcal{O}(B), \mathcal{C}(B), D(B))$ for affected blocks
Assign confirmation grade:
if $\mathcal{F}(B) \geq \tau_{\text{settle}}$ **then**
    Grade 3: Deep Settlement
else if $\mathcal{O}(B) \geq \tau_{\text{ord}} \wedge \mathcal{C}(B) \geq c_{\text{threshold}}$ **then**
    Grade 2: Strong Confirmation
else if $\mathcal{O}(B) \geq \tau_{\text{ord}}$ **then**
    Grade 1: Tentative Ordering
else
    No confirmation
end if
Update $\Sigma_{\text{fin}}(B_{\text{new}})$ and affected blocks

// — **Propagation decision** —
Compute $\Phi(S)$ for candidate mining templates
Select template maximising $U_{\text{order}} - \lambda R_{\text{closure}}$

A Relationship to GhostDAG's K Parameter

GhostDAG's security parameter K is derived from the network delay bound D and block production rate f under UDBM assumptions [6]:

$$K = f(2Df,\, \delta) \tag{26}$$

where δ is the tail probability that the anticone of a block exceeds K. The derivation assumes that blocks are accepted within delay D.

Under CBM, the effective delay is $\Delta_{\text{actual}} = D + \Delta_{\text{proc}}$, which may exceed D. A naïve adaptation would replace D with $D_{\text{eff}} = D + \mathbb{E}[\Delta_{\text{proc}}]$:

$$K_{\text{naive}} = f(2D_{\text{eff}}f,\, \delta) \tag{27}$$

However, this approach has two problems:

1. $\mathbb{E}[\Delta_{\text{proc}}]$ is not a fixed constant but depends on network conditions and attack intensity, making K_{naive} unstable.
2. Changing K is a hard consensus change—all nodes must agree on the same K—but D_{eff} varies across nodes under LP attack.

PHASMDAG resolves this differently. The closure layer provides a *per-block* modulation of effective delay exposure, without changing the global K:

- The K parameter remains a consensus constant derived from the optimistic D (as in current GhostDAG).
- The closure level $\mathcal{C}(B)$ modulates the *effective weight* of each block within the K-cluster, rather than changing K itself.
- Blocks with $\mathcal{C}(B) < 1$ contribute less to blue-work accumulation, which is mathematically equivalent to a locally higher effective K for those blocks, without requiring consensus-level parameter changes.

This design choice is deliberate: it keeps the consensus rule simple and stable (K is fixed) while achieving the security benefit of adaptive delay awareness through the closure layer. The tradeoff is that PHASMDAG's blue work grows more slowly under LP attack than GhostDAG's would under UDBM, but the growth is more *faithful*—it reflects actual dependency maturity rather than optimistic assumptions.

Table 7: Parameter handling comparison

Approach	**Consensus K**	**Delay handling**
GhostDAG (UDBM)	$K = f(2Df, \delta)$	D assumed uniform
GhostDAG + adaptive K	$K = f(2D_{\text{eff}}f, \delta)$	Consensus risk: nodes disagree on D_{eff}
PHASMDAG	$K = f(2Df, \delta)$ (unchanged)	Per-block $\mathcal{C}(B)$ modulates effective weight